```
!pip install -U transformers datasets accelerate -q

import torch
import math
from transformers import (
    GPT2LMHeadModel,
    GPT2Tokenizer,
    Trainer,
    TrainingArguments,
    DataCollatorForLanguageModeling,
    set_seed
)
from datasets import Dataset

set_seed(42)

device = "cuda" if torch.cuda.is_available() else "cpu"
print("Device:", device)

def generate_text(model, tokenizer, prompt, max_length=100):
    model.eval()
    inputs = tokenizer.encode(prompt, return_tensors="pt").to(device)

    with torch.no_grad():
        output = model.generate(
            inputs,
            max_length=max_length,
            temperature=0.8,
            top_k=50,
            do_sample=True,
            pad_token_id=tokenizer.eos_token_id
        )

    return tokenizer.decode(output[0], skip_special_tokens=True)
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.1/10.1 MB 99.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 527.0/527.0 kB 43.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 47.6/47.6 MB 20.1 MB/s eta 0:00:00
Device: cuda
```

```
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2").to(device)

tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = tokenizer.eos_token_id

# Baseline
review_prompts = [
    "This product is",
    "I bought this phone and",
    "The quality of this item"
]

baseline = {}
print("=== BASELINE ===")
for p in review_prompts:
    baseline[p] = generate_text(model, tokenizer, p)
    print(f"\n{p}\n{baseline[p]}")

# Dataset
corpus = [
    "this phone has an amazing battery life and the camera quality is outstanding for the price.",
    "i bought this laptop for college and it handles all my assignments and coding projects perfectly.",
    "the sound quality of these headphones is incredible with deep bass and clear vocals.",
    "this smartwatch tracks my steps accurately and the heart rate monitor is very reliable.",
    "great wireless earbuds with noise cancellation that blocks out all background sound.",
    "the keyboard feels very comfortable for long typing sessions and the backlight is a nice touch.",
    "this portable charger saved me during travel and it charges my phone three times on a single charge.",
    "the tablet screen is bright and colorful which makes watching movies a great experience.",
    "i love this fitness tracker because it motivates me to reach my daily exercise goals.",
    "this bluetooth speaker is compact but delivers surprisingly loud and clear audio.",
    "the delivery was fast and the product was packed securely with no damage at all.",
    "excellent value for money and the build quality feels premium despite the affordable price.",
    "the customer service team was very helpful when i had questions about the product features.",
    "this camera takes stunning photos in low light and the video recording quality is very smooth.",
    "i have been using this product for three months and it still works perfectly like day one.",
    "the design is sleek and modern and it looks great on my desk next to my other gadgets.",
    "easy to set up right out of the box and the instructions were clear and simple to follow.",
    "highly recommend this product to anyone looking for quality and reliability at a fair price.",
    "the software updates keep adding new features which makes this purchase even more worthwhile.",
    "best purchase i made this year and i would definitely buy from this brand again."
```

```
    best purchase I made this year and I would definitely buy from this brand again.
]

dataset = Dataset.from_dict({"text": corpus})

tokenized = dataset.map(
    lambda x: tokenizer(x["text"], truncation=True, padding="max_length", max_length=128),
    batched=True,
    remove_columns=["text"]
)

split = tokenized.train_test_split(test_size=0.15, seed=42)

data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)

training_args = TrainingArguments(
    output_dir="./gpt2-reviews",
    num_train_epochs=5,   # faster for colab
    per_device_train_batch_size=4,
    learning_rate=5e-5,
    weight_decay=0.01,
    warmup_steps=50,
    eval_strategy="epoch",
    logging_steps=10,
    save_strategy="no",
    fp16=torch.cuda.is_available(),
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=split["train"],
    eval_dataset=split["test"],
    data_collator=data_collator
)

trainer.train()

# Evaluation
eval_res = trainer.evaluate()
print("\nPerplexity:", math.exp(eval_res["eval_loss"]))

# Comparison
print("\n=== FINE-TUNED ===")
for p in review_prompts:
    ft = generate_text(model, tokenizer, p)
    print(f"\n{p}")
    print("Baseline:", baseline[p][:100])
    print("Fine-tuned:", ft[:100])
```

```
Loading weights: 100%                                    148/148 [00:00<00:00, 415.71it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                  | Status     |  |
---------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
=== BASELINE ===

This product is
This product is designed to be used in conjunction with any other device and is intended for use in conjunction with any oth

The following devices may be inserted into any such device, even if the device does not have a user interface or an open dev

a) any personal computer;

b) other personal computer or computer-based mobile device;

c) any computer-based home computer; and

d) any personal computer-based network-

I bought this phone and
I bought this phone and started working on it. I'm quite pleased with the way it performs, but this phone has the same desig

The quality of this item
The quality of this item is the same as that of any other item on the item page.
Map: 100%                                    20/20 [00:00<00:00, 504.61 examples/s]
`loss_type=None` was set in the config but it is unrecognized. Using the default loss: `ForCausalLMLoss`.
[25/25 00:02, Epoch 5/5]
```

Epoch	Training Loss	Validation Loss
1	No log	3.348063
2	4.057265	3.247350
3	4.057265	3.106139
4	3.319935	2.981021
5	3.319935	2.881039

```
[1/1 : < :]

Perplexity: 17.832798622998247

=== FINE-TUNED ===

This product is
Baseline: This product is designed to be used in conjunction with any other device and is intended for use in
Fine-tuned: This product is free of lead and has no odor or irritation. The product is tested on a 5K HMD in a r

I bought this phone and
Baseline: I bought this phone and started working on it. I'm quite pleased with the way it performs, but this
Fine-tuned: I bought this phone and I love the features of the camera. I'm impressed by how well the colors matc

The quality of this item
Baseline: The quality of this item is the same as that of any other item on the item page.
Fine-tuned: The quality of this item is very good and is very accurate in practice. The knife is fine for beginn
```

```
tokenizer2 = GPT2Tokenizer.from_pretrained("gpt2")
model2 = GPT2LMHeadModel.from_pretrained("gpt2").to(device)

tokenizer2.pad_token = tokenizer2.eos_token
model2.config.pad_token_id = tokenizer2.eos_token_id

recipe_prompts = [
    "To make butter chicken",
    "For pasta carbonara",
    "To prepare chocolate cake"
]

baseline2 = {}
print("=== BASELINE RECIPES ===")
for p in recipe_prompts:
    baseline2[p] = generate_text(model2, tokenizer2, p)
    print(f"\n{p}\n{baseline2[p]}")

recipes = [
    "to make butter chicken start by marinating chicken pieces in yogurt with spices.",
    "heat butter and fry onions until golden then add ginger garlic paste.",
    "add tomato puree and cook until oil separates.",
    "add chicken and cook until tender.",
```

```
    "finish with cream and serve hot.",
    "boil pasta until al dente.",
    "fry pancetta until crispy.",
    "mix eggs and cheese.",
    "combine pasta with mixture to form creamy sauce.",
    "serve immediately."
]

dataset2 = Dataset.from_dict({"text": recipes})

tok2 = dataset2.map(
    lambda x: tokenizer2(x["text"], truncation=True, padding="max_length", max_length=128),
    batched=True,
    remove_columns=["text"]
)

split2 = tok2.train_test_split(test_size=0.15, seed=42)

collator2 = DataCollatorForLanguageModeling(tokenizer=tokenizer2, mlm=False)

args2 = TrainingArguments(
    output_dir="./gpt2-recipes",
    num_train_epochs=5,
    per_device_train_batch_size=4,
    learning_rate=5e-5,
    weight_decay=0.01,
    warmup_steps=50,
    eval_strategy="epoch",
    logging_steps=10,
    save_strategy="no",
    fp16=torch.cuda.is_available(),
)

trainer2 = Trainer(
    model=model2,
    args=args2,
    train_dataset=split2["train"],
    eval_dataset=split2["test"],
    data_collator=collator2
)

trainer2.train()

eval2 = trainer2.evaluate()
print("\nPerplexity:", math.exp(eval2["eval_loss"]))

print("\n=== FINE-TUNED RECIPES ===")
for p in recipe_prompts:
    ft = generate_text(model2, tokenizer2, p)
    print(f"\n{p}")
    print("Baseline:", baseline2[p][:100])
    print("Fine-tuned:", ft[:100])
```

```
Loading weights: 100%                                                    148/148 [00:00<00:00, 504.20it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                  | Status     |  |
---------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
=== BASELINE RECIPES ===

To make butter chicken
To make butter chicken, heat oven to 350°F. In a bowl, whisk egg whites, milk, sugar and flour together so that mixture is s

In a large bowl, whisk together flour until well combined. Pour in eggs, cornstarch, salt and pepper and mix. Slice into abo

For pasta carbonara
For pasta carbonara (spaghetti) with olive oil, tomato paste, garlic powder, parsley, herbs and spices.

Chocolate chip cookies (Sugar cookie) with chocolate chips and sugar, almonds, cacao nibs, sugar, cinnamon, nutmeg and more.
```